

Ball machine

Madina je izumila mašinu s kuglicama kako bi zabavila učesnike IOI-ja. Unutrašnja struktura mašine je sljedeća:

- Mašina se sastoji od N **čvorova**, indeksiranih od 0 do $N - 1$. Čvor $N - 1$ naziva se **korijen**.
- Za svaki čvor u takav da je $0 \leq u < N - 1$, **roditelj** čvora u je čvor $P[u]$ za koji važi $P[u] > u$, a čvor u je **dijete** čvora $P[u]$. Korijen nema roditelja. Čvor koji nema djece naziva se **list**.

Vi **ne znate** vrijednost N niti roditelja $P[u]$ bilo kojeg čvora u . Umjesto toga, Madina vam saopštava M , broj listova, i činjenicu da su indeksi listova $0, 1, \dots, M - 1$. Vaš zadatak je odrediti unutrašnju strukturu mašine upravljajući njome.

Svaki čvor mašine može sadržavati najviše jednu **kuglicu**, a svaka kuglica ima **vrijednost** koja je nenegativan cijeli broj. Kažemo da čvor ima vrijednost x ako sadrži kuglicu vrijednosti x . Čvor je **zauzet** ako sadrži kuglicu; u suprotnom je **slobodan**. Na početku je svaki čvor slobodan.

Možete izvršavati dvije vrste operacija:

- **insert**(U, X): pokušava umetnuti novu kuglicu vrijednosti X u list U .
 - Ako je list U zauzet, operacija vraća `false` bez umetanja kuglice.
 - Ako je list U slobodan, kuglica se postavlja u čvor U . Zatim se kuglica uzastopno pomjera do roditelja trenutnog čvora sve dok je taj roditelj slobodan. Pomjeranje se zaustavlja kada kuglica stigne do korijena ili do čvora čiji je roditelj zauzet. Operacija vraća `true`.
- **collect**(): ako je korijen slobodan, što znači da je mašina prazna, `collect` vraća prazan niz. U suprotnom, rekurzivna funkcija `traverse` opisana ispod prikuplja vrijednosti svih kuglica koje se trenutno nalaze u mašini u niz S . Na početku je niz S prazan i poziva se `traverse(N - 1)`.

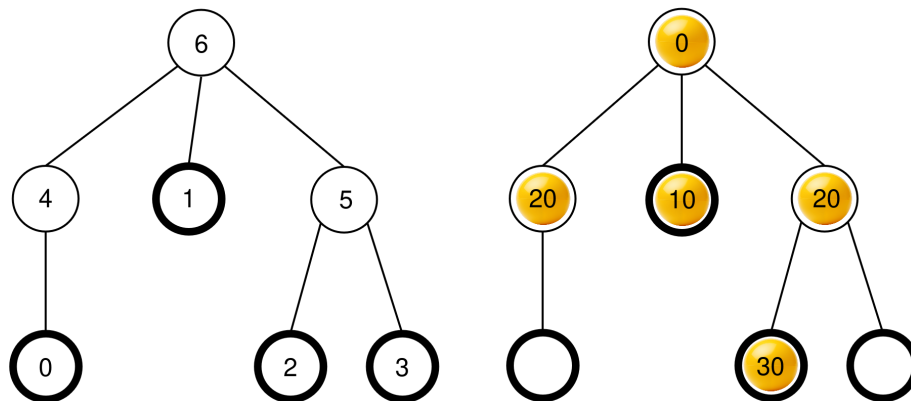
```

traverse(u):
    dodaj vrijednost kuglice u čvoru u na kraj niza S
    neka je c[u] lista zauzete djece čvora u
    sortiraj c[u] neopadajuće prema vrijednostima
    kuglica u tim čvorovima (ako više čvorova ima istu
    vrijednost, mogu se pojaviti bilo kojim redoslijedom)
    za svaki v u c[u]:
        traverse(v)

```

Nakon završetka ove funkcije, **sve kuglice se uklanjaju** iz mašine, a `collect` vraća S .

Na primjer, razmotrimo mašinu sa $N = 7$ čvorova i nizom roditelja $P = [4, 6, 5, 5, 6, 6]$. Slika lijevo prikazuje praznu mašinu sa indeksima čvorova, dok slika desno prikazuje jedan mogući raspored kuglica nakon određenog niza operacija `insert` (taj niz je naveden u odjeljku Primjer).



Kada se pozove `collect()`, korijen, odnosno čvor 6, je zauzet, pa se S inicijalizira kao $S = []$ i poziva se `traverse(6)`.

`traverse(6)`: čvor 6 ima vrijednost 0; dodavanjem te vrijednosti u S dobijamo $S = [0]$. Djeca čvora 6 su čvorovi 4, 1 i 5, i sva tri su zauzeta. Njihove vrijednosti su redom 20, 10 i 20. Sortiranjem neopadajuće dobijamo $c[6] = [1, 5, 4]$ (primijetite da bi $c[6] = [1, 4, 5]$ bilo ispravno, jer čvorovi 4 i 5 imaju istu vrijednost). Funkcija zatim poziva `traverse` za svaki čvor u $c[6]$, tim redoslijedom.

- **`traverse(1)`:** čvor 1 ima vrijednost 10; dodavanjem u S dobijamo $S = [0, 10]$. Čvor 1 nema zauzete djece, pa se ovaj poziv završava.
- **`traverse(5)`:** čvor 5 ima vrijednost 20; dodavanjem u S dobijamo $S = [0, 10, 20]$. Čvor 5 ima jedno zauzeto dijete, čvor 2, pa je $c[5] = [2]$ i funkcija poziva `traverse(2)`.
 - **`traverse(2)`:** čvor 2 ima vrijednost 30; dodavanjem u S dobijamo $S = [0, 10, 20, 30]$. Čvor 2 nema zauzete djece, pa se ovaj poziv završava.
 - Izvršavanje se vraća u `traverse(5)`, koji je obradio svu djecu iz $c[5]$, pa se njegov poziv završava.

- **traverse(4)**: čvor 4 ima vrijednost 20; dodavanjem u S dobijamo $S = [0, 10, 20, 30, 20]$. Čvor 4 nema zauzete djece, pa se ovaj poziv završava.

Svi pozivi su završeni i nema više čvorova za obradu. Na kraju se sve kuglice uklanjaju iz mašine i `collect` vraća niz $S = [0, 10, 20, 30, 20]$.

Vaš zadatak je odrediti broj čvorova N i prijaviti niz roditelja $R = [R[0], R[1], \dots, R[N-2]]$ koji opisuje strukturu mašine, ali uz moguće drugačije označavanje čvorova koji nisu ni listovi ni korijen. Formalno, prijavljeni niz R smatra se ispravnim ako je moguće svakom čvoru u mašine dodijeliti različitu oznaku $L[u]$, pri čemu je $0 \leq L[u] < N$, tako da:

- $L[u] = u$ za sve $0 \leq u < M$ i za $u = N-1$, i
- $L[P[u]] = R[L[u]]$ za sve $0 \leq u < N-1$.

Nije potrebno da važi $R[u] > u$. Mašina **mora biti prazna** kada prijavite svoje rješenje.

Neka je K broj izvršenih operacija `collect`, a B najveća vrijednost bilo koje kuglice među svim pozivima `insert`. Neka je $C = K + B$. Tada C **ne smije biti veće od 1000**. Vaš broj bodova u nekim podzadacima zavisi od vrijednosti C .

Detalji implementacije

Potrebno je implementirati sljedeću funkciju.

```
std::vector<int> find_structure(int M)
```

- M : broj listova u mašini.
- Funkcija se poziva tačno jednom za svaki testni slučaj.
- Funkcija mora vratiti niz $R = [R[0], R[1], \dots, R[N-2]]$ dužine $N-1$, koji predstavlja ispravan niz roditelja.

Za interakciju s mašinom vaša funkcija može pozivati sljedeće dvije funkcije.

Prva funkcija je:

```
bool insert(int U, int X)
```

- U : indeks lista u koji treba umetnuti kuglicu. Mora važiti $0 \leq U < M$.
- X : cjelobrojna vrijednost kuglice. Mora važiti $0 \leq X \leq 1000$.
- Funkcija vraća `true` ako je kuglica uspješno umetnuta, ili `false` ako je list U već bio zauzet.
- Ova funkcija može se pozvati najviše 500 000 puta.

Druga funkcija je:

```
std::vector<int> collect()
```

- Prikuplja vrijednosti svih umetnutih kuglica, prazni mašinu i vraća prikupljeni niz S .

Ako u bilo kojem trenutku tokom izvršavanja programa vrijednost C pređe 1000, vaše rješenje će dobiti presudu `Output isn't correct: Too many resources used`.

Struktura mašine je fiksirana prije poziva `find_structure`. Ocjenjivač je **determinističan**, u smislu da će, ako ga pokrenete dva puta i oba pokretanja izvrše isti niz operacija, pozivi `collect` vratiti iste nizove.

Ograničenja

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

Podzadaci

Podzadatak	Bodovi	Dodatna ograničenja
1	5	Korijen ima tačno M djece.
2	10	$M \leq 3$
3	25	$N \leq 200, M \leq 45$
4	60	Nema dodatnih ograničenja.

U podzadacima 3 i 4, broj bodova zavisi od vrijednosti C na sljedeći način.

Podzadatak 3 (25 bodova)

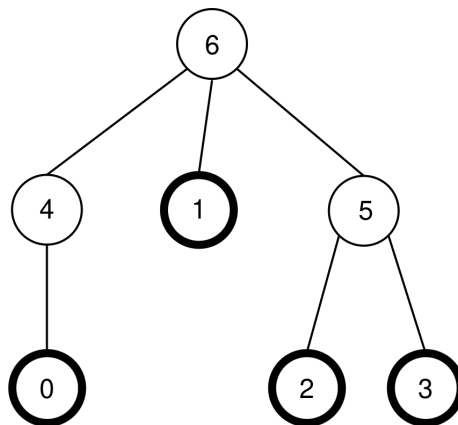
Ograničenje	Bodovi
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

Podzadatak 4 (60 bodova)

Ograničenje	Bodovi
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Primjer

Razmotrimo istu mašinu kao u opisu zadatka, dakle $N = 7$, $M = 4$ i $P = [4, 6, 5, 5, 6, 6]$. Mašina je ponovo prikazana na sljedećoj slici:



Ocjenjivač poziva:

```
find_structure(4)
```

Funkcija može izvršiti sljedeći niz poziva:

1. **insert(0, 0)**: list 0 je slobodan, pa se kuglica vrijednosti 0 postavlja u list 0. Čvor $P[0] = 4$ je slobodan, pa se kuglica pomjera u čvor 4. Čvor $P[4] = 6$ je slobodan, pa se kuglica pomjera u čvor 6. Pošto je čvor 6 korijen, pomjeranje se zaustavlja. Poziv vraća `true`.
2. **insert(3, 20)**: list 3 je slobodan, pa se kuglica vrijednosti 20 postavlja u list 3. Čvor $P[3] = 5$ je slobodan, pa se kuglica pomjera u čvor 5. Pošto je $P[5] = 6$ zauzet, pomjeranje se zaustavlja. Poziv vraća `true`.
3. **insert(1, 10)**: list 1 je slobodan, pa se kuglica vrijednosti 10 postavlja u list 1. Pošto je $P[1] = 6$ zauzet, kuglica ostaje u čvoru 1. Poziv vraća `true`.
4. **insert(2, 30)**: list 2 je slobodan, pa se kuglica vrijednosti 30 postavlja u list 2. Pošto je $P[2] = 5$ zauzet, kuglica ostaje u čvoru 2. Poziv vraća `true`.

5. `insert(1, 25)`: list 1 je zauzet, pa se kuglica ne postavlja u list 1. Poziv vraća `false`.

6. `insert(0, 20)`: list 0 je slobodan, pa se kuglica vrijednosti 20 postavlja u list 0. Čvor $P[0] = 4$ je slobodan, pa se kuglica pomjera u čvor 4. Pošto je $P[4] = 6$ zauzet, pomjeranje se zaustavlja. Poziv vraća `true`.

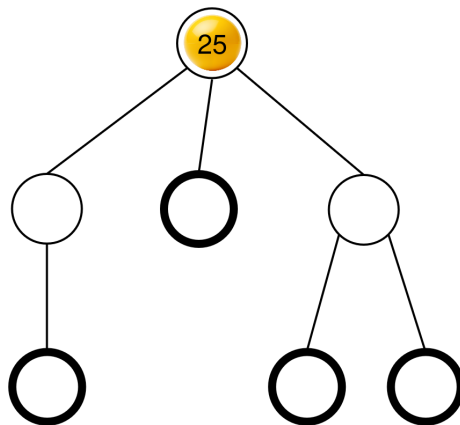
Dobijeni raspored kuglica već je prikazan u opisu zadatka.

Zatim funkcija može pozvati:

```
collect()
```

Izvršavanje ovog poziva već je objašnjeno u opisu zadatka, pa poziv vraća $S = [0, 10, 20, 30, 20]$. Nakon toga je mašina ponovo prazna.

Zatim funkcija može pozvati `insert(2, 25)`. List 2 je slobodan, pa se kuglica vrijednosti 25 postavlja u list 2. Kuglica se zatim pomjera u čvor 5, a potom u čvor 6, gdje se zaustavlja. Poziv vraća `true`. Dobijeni raspored kuglica prikazan je na sljedećoj slici.



Na kraju funkcija može pozvati `collect()`, koji vraća $S = [25]$ i prazni mašinu.

Postoje dva ispravna niza roditelja koja `find_structure` može vratiti: $R = P = [4, 6, 5, 5, 6, 6]$ (što odgovara označavanju $L = [0, 1, 2, 3, 4, 5, 6]$), i $R = [5, 6, 4, 4, 6, 6]$ (što odgovara označavanju $L = [0, 1, 2, 3, 5, 4, 6]$). U ovom primjeru je $K = 2$ i $B = 30$, pa je $C = 32$.

Primjer ocjenjivača

Format ulaza:

```
N M
P[0] P[1] P[2] ... P[N-2]
```

Format izlaza:

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

Ovdje je Q dužina niza R koji vraća `find_structure`.